

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÀI TẬP LỚN

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Đề tài

NGHIÊN CỨU VÀ XÂY DỰNG THUẬT TOÁN TÌM KIẾM TỪ HAI PHÍA

Sinh viên thực hiện: BÙI MINH QUÂN 0212468

NGUYỄN PHÚC THANH

NINH THỊ THANH VÂN

PHẠM THÙY DUNG

Lớp 68CS3

Giảng viên hướng dẫn: TS. Phạm Hồng Phong

Hà Nội, 9-2025

LỜI NÓI ĐẦU

Trong lĩnh vực Trí tuệ Nhân tạo và Khoa học Máy tính, bài toán tìm kiếm đường đi (search problems) là một trong những vấn đề nền tảng và cốt lõi. Các bài toán này xuất hiện trong vô số ứng dụng thực tế, từ những bài toán cụ thể như tìm đường đi ngắn nhất trên bản đồ, giải các câu đố (puzzle) như Rubik hay 8-puzzle, cho đến những vấn đề phức tạp hơn như lập kế hoạch hành động cho robot hay thậm chí là chẩn đoán y khoa. Về bản chất, những bài toán này có thể được mô hình hóa thành việc duyệt qua một **không gian trạng thái** - một đồ thị mà trong đó các nút đại diện cho các trạng thái có thể có và các cạnh đại diện cho các bước chuyển đổi giữa các trạng thái đó. Mục tiêu cuối cùng là tìm ra một đường đi tối ưu (về chi phí hoặc độ dài) từ trạng thái bắt đầu (s) đến trạng thái đích (g).

Để giải quyết các bài toán này, nhiều thuật toán tìm kiếm kinh điển đã được phát triển. Các thuật toán phổ biến như Tìm kiếm theo Chiều Rộng (BFS - Breadth-First Search), Tìm kiếm theo Chiều Sâu (DFS - Depth-First Search), Tìm kiếm Chi phí Đều (UCS - Uniform Cost Search) hay Tìm kiếm A* là những công cụ mạnh mẽ. Tuy nhiên, điểm chung của các thuật toán này là chúng đều là **tìm kiếm đơn hướng** (unidirectional). Nghĩa là, cây tìm kiếm chỉ được phát triển từ một nút duy nhất - nút gốc (s - trạng thái bắt đầu) và mở rộng dần ra cho đến khi may mắn chạm tới được trạng thái đích.

Chính sự điều này trở thành điểm yếu chí mạng khi không gian tìm kiếm trở nên quá lớn. Chẳng hạn, thuật toán BFS, mặc dù luôn tìm được đường đi ngắn nhất (nếu tồn tại), lại có độ phức tạp thời gian và không gian trong trường hợp xấu nhất là $O(b^d)$, trong đó b là hệ số phân nhánh (branching factor - số trạng thái kế thừa trung bình) và d là độ sâu của lời giải. Độ phức tạp theo cấp số mũ này đồng nghĩa với việc lượng bộ nhớ và thời gian cần thiết để tìm kiếm sẽ tăng vọt một cách khủng khiếp khi d chỉ cần tăng lên một chút. Các thuật toán này giống như một người đứng giữa một "đại dương" các trạng thái mênh mông và cố gắng dò tìm từng mét vuông một mà không biết đích đang ở hướng nào, khiến chúng trở nên kém hiệu quả một cách tuyệt vọng trong các bài toán quy mô lớn.

Trước thách thức đó, **Tìm kiếm Hai chiều (Bidirectional Search)** đã ra đời như một chiến lược tối ưu hóa thông minh, mang tính đột phá. Thay vì "bơi" một mình từ điểm xuất phát, phương pháp này vận dụng nguyên tắc đồng thời xây dựng **hai cây tìm kiếm**: một cây bắt đầu từ trạng thái xuất phát và một cây khởi nguồn từ chính trạng thái đích. Bằng cách tận dụng tính đối xứng của đường đi, nó giảm thiểu đáng kể độ sâu tìm kiếm cần thiết. Nếu một thuật toán đơn hướng phải duyệt đến độ sâu d , thì Bidirectional Search mỗi phía chỉ cần duyệt sâu khoảng $d/2$. Điều này chuyển hóa độ phức tạp tính toán từ $O(b^d)$ thành $O(b^{d/2})$, một sự cải thiện theo cấp số nhân, giúp rút ngắn thời gian tìm kiếm một cách ngoạn mục. Để đạt được sự tối ưu này, điều kiện tiên quyết là phải sử dụng các chiến lược tìm kiếm có hệ thống, chẳng hạn như BFS, cho cả hai hướng.

Xuất phát từ bối cảnh và tiềm năng ứng dụng to lớn đó, bài tiểu luận này được thực hiện với mục tiêu phân tích một cách toàn diện về Bidirectional Search. Cụ thể, bài viết sẽ đi sâu vào:

- Nguyên lý hoạt động và cơ sở lý thuyết của phương pháp.
- Ưu điểm vượt trội và những hạn chế tiềm tàng.
- Các biến thể chính của thuật toán.
- Ứng dụng thực tế của nó trong các lĩnh vực khác nhau của Khoa học Máy tính.

Mục lục

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	i
DANH MỤC HÌNH VẼ	ii
DANH MỤC BẢNG	iii
TÓM TẮT BÀI TẬP LỚN	iv
CHƯƠNG 1. CHƯƠNG MỞ ĐẦU	1
1.1 Phương pháp chính để triển khai tìm kiếm 2 chiều	1
1.2 Mục tiêu của bài tập	1
1.2.1 Về Lý thuyết nền tảng	1
1.2.2 Triển khai và thực nghiệm	1
1.2.3 Về đánh giá và Tổng kết	2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	3
2.1 Định nghĩa và Cơ chế hoạt động	3
2.1.1 Định nghĩa	3
2.1.2 Cơ chế hoạt động	3
2.1.3 Ví dụ minh họa thuật toán	3
2.2 Yêu cầu Thuật toán cơ sở và Biến thể	4
2.2.1 Thuật toán BFS trong Bidirectional Search	4
2.2.2 Tại sao không nên sử dụng DFS trong Bidirectional	4
2.2.3 Một số biến thể của Bidirectional	5
2.3 Phân tích độ phức tạp của thuật toán, Ưu và Nhược điểm khi so sánh với các thuật toán khác	5
2.3.1 Phân tích độ phức tạp thuật toán	5
2.3.2 So sánh với các Thuật toán Tìm kiếm Khác	6
2.3.3 Ưu điểm và Nhược điểm	6
2.3.4 Kết luận	6
2.4 Nhược điểm và thách thức kỹ thuật	6
2.5 Ứng dụng và hạn chế trong thực tế	6
2.5.1 Ứng dụng trong thực tế	7
2.5.2 Hạn chế trong thực tế	7
2.6 Kết luận	7

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Danh sách hình vẽ

Hình 2. 1 Ví dụ minh họa cho thuật toán 3

Danh sách bảng

Bảng 2. 1	So sánh các biến thể của Tìm kiếm Hai chiều	5
Bảng 2. 2	So sánh các thuật toán tìm kiếm	6

TÓM TẮT BÀI TẬP LỚN

Trình bày mục đích và các kết luận quan trọng nhất của bài

CHƯƠNG 1. CHƯƠNG MỞ ĐẦU

Trong lĩnh vực Khoa học Máy tính, bài toán tìm kiếm đường đi (pathfinding) hoặc tìm kiếm trạng thái (state search) trên các cấu trúc đồ thị là một bài toán nền tảng với vô số ứng dụng thực tiễn, từ trí tuệ nhân tạo và robot cho đến hệ thống định tuyến mạng. Để giải quyết bài toán này, bên cạnh các thuật toán tìm kiếm kinh điển theo một hướng như Dijkstra hay A* (tìm kiếm từ nút nguồn đến nút đích), Thuật toán Tìm kiếm Hai Chiều (Bidirectional Search) đã được đề xuất như một phương pháp hiệu quả nhằm giảm thiểu đáng kể không gian và thời gian tìm kiếm. Ý tưởng cốt lõi của phương pháp này là khởi chạy đồng thời hai cuộc tìm kiếm: một từ nguồn (start) và một từ đích (goal), với kỳ vọng hai "mặt trận" này sẽ gặp nhau ở đâu đó tại trung tâm của đồ thị, từ đó cắt giảm một nửa độ sâu tìm kiếm và giảm hàm mũ số lượng nút phải duyệt.

1.1 Phương pháp chính để triển khai tìm kiếm 2 chiều

- **Bidirectional BFS (Breadth-First Search):** Sử dụng hai hàng đợi (queues) để triển khai tìm kiếm theo chiều rộng từ cả hai phía.
- **Bidirectional A*:** Kết hợp heuristic để hướng dẫn cả hai cuộc tìm kiếm, nâng cao hiệu suất so với BFS thuần túy.
- **Các kỹ thuật đồng bộ hóa và quyết định điểm gặp nhau:** Đây là thách thức lớn nhất, đòi hỏi các chiến lược như luân phiên tìm kiếm (alternating) hoặc dự đoán điểm gặp nhau dựa trên hàm heuristic để đảm bảo hai phía gặp nhau một cách tối ưu.

1.2 Mục tiêu của bài tập

Hiểu một cách sâu sắc và toàn diện về thuật toán Tìm kiếm Hai Chiều, từ cơ sở lý thuyết đến các thách thức trong triển khai thực tế, đặc biệt là trong môi trường song song. Từ đó, xây dựng một chương trình minh họa hiệu quả, tiến hành đánh giá, so sánh và đưa ra kết luận về ưu, nhược điểm của phương pháp này.

1.2.1 Về Lý thuyết nền tảng:

- Trình bày được khái niệm và cơ chế hoạt động cốt lõi của Bidirectional Search: cách thức hai cuộc tìm kiếm từ nguồn và đích được khởi chạy, mở rộng và quyết định điểm gặp nhau.
- Phân tích một cách hệ thống các ưu điểm và nhược điểm chính của phương pháp này so với tìm kiếm một chiều truyền thống (ví dụ: tiết kiệm không gian tìm kiếm nhưng phức tạp trong đồng bộ).
- Mô tả chi tiết thuật toán cơ sở (Bidirectional BFS), bao gồm mã giả và các yếu tố kỹ thuật then chốt như cấu trúc dữ liệu sử dụng (hàng đợi, tập hợp đã duyệt), chiến lược luân phiên, và điều kiện dừng.
- Tìm hiểu các ứng dụng điển hình của thuật toán trong các bài toán thực tế như tìm đường đi, giải trò chơi, và truy vấn trong cơ sở dữ liệu đồ thị.
- Chỉ ra được các hạn chế vốn có của thuật toán (ví dụ: khó áp dụng cho đồ thị có hướng hoặc khi không xác định được trạng thái đích rõ ràng).

1.2.2 Triển khai và thực nghiệm

- Triển khai thành công phiên bản tuần tự của thuật toán Bidirectional BFS dựa trên mã giả và lý thuyết đã nghiên cứu.
- Xây dựng bộ dữ liệu thử nghiệm phù hợp (các đồ thị mẫu với các kích thước và mức độ phức tạp khác nhau) để đánh giá hiệu năng.

- Tiến hành thực nghiệm, đo đạc và phân tích kết quả một cách khách quan dựa trên các thước đo như thời gian thực thi, số lượng đỉnh được duyệt, và tốc độ tăng tốc (speedup) khi so sánh giữa hai phiên bản tuần tự và song song.

1.2.3 Về đánh giá và Tổng kết

- Đưa ra nhận định và kết luận về hiệu quả của việc áp dụng song song hóa vào thuật toán Bidirectional Search dựa trên kết quả thực nghiệm.
- Chỉ ra các hạn chế của giải pháp đã triển khai và đề xuất các hướng phát triển, mở rộng trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Tìm kiếm hai chiều (Bidirectional Search) là một phương pháp tìm kiếm trong không gian trạng thái, được xếp vào nhóm tìm kiếm không có thông tin (tìm kiếm mù). Cơ sở lý thuyết của nó là nguyên tắc đồng thời xây dựng hai cây tìm kiếm: một cây bắt đầu từ trạng thái xuất phát và một cây bắt đầu từ trạng thái đích. Phương pháp này tận dụng tính đối xứng của đường đi, nhằm mục đích giảm độ sâu tìm kiếm cần thiết xuống còn $d/2$ (trong đó d là độ sâu lời giải nông nhất), qua đó tối ưu hóa đáng kể độ phức tạp tính toán thành $O(b^{d/2})$ so với tìm kiếm một chiều là $O(b^d)$. Để đạt được tính tối ưu này, điều kiện tiên quyết là phải sử dụng tìm kiếm theo chiều rộng (BFS) cho cả hai hướng.

2.1 Định nghĩa và Cơ chế hoạt động

Sau đây là một bài lưu ý khi làm đồ án cần nhớ nhé:

2.1.1 Định nghĩa

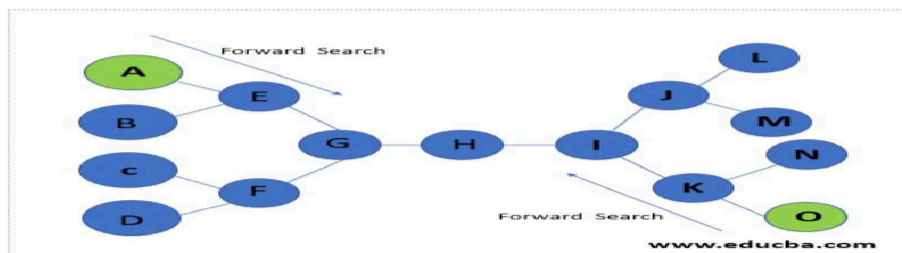
Tìm kiếm hai chiều là phương pháp xây dựng đồng thời hai cây tìm kiếm có nút gốc là trạng thái xuất phát và trạng thái đích. Quá trình tìm kiếm kết thúc khi hai quá trình gặp nhau tại một đỉnh chung (điểm giao nhau), kết nối hai nửa đường đi để tạo thành đường đi đầy đủ từ nguồn đến đích.

- Tìm kiếm xuôi (forward search) từ đỉnh nguồn đến đỉnh đích
- Tìm kiếm ngược (backward search) từ đỉnh đích về đỉnh nguồn

2.1.2 Cơ chế hoạt động

- Khởi tạo: hai tập Frontier (Open List) cho Start và Goal.
- Mở rộng luân phiên: mỗi bước mở rộng một tầng từ cả hai phía.
- Kiểm tra giao nhau: sau mỗi bước, kiểm tra xem có nút nào nằm trong cả hai tập Frontier.
- Ghép lời giải: nếu có, dừng lại và nối hai đường đi.
- Điều kiện dừng: khi một Frontier rỗng mà không tìm được đường đi $\rightarrow$ không có lời giải.

2.1.3 Ví dụ minh họa thuật toán



Hình 2. 1 Ví dụ minh họa cho thuật toán

Hình 2. 1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Tất cả hình vẽ phải được để cap đen trong phạm vi dung và phải được để cap đen riêng pohan nội dung và fjd cphan tích

2.2 Yêu cầu Thuật toán cơ sở và Biến thể

2.2.1 Thuật toán BFS trong Bidirectional Search

Tim kiếm theo Chiều Rộng (BFS) là một trong những thuật toán tìm kiếm không có thông tin (tìm kiếm mù) cơ bản nhất. Nguyên tắc hoạt động của BFS là ưu tiên mở rộng nút nông nhất (gần nút gốc nhất) trước. Thuật toán đảm bảo tính đầy đủ (luôn tìm ra lời giải nếu tồn tại) và tính tối ưu (luôn tìm ra lời giải có độ sâu nhỏ nhất) nếu giá thành đường đi giữa các nút là như nhau. BFS được triển khai bằng cách sử dụng hàng đợi FIFO để lưu trữ các nút biên (nút mở).

Trong bối cảnh của Tim kiếm hai chiều, việc sử dụng BFS là **điều kiện bắt buộc** khi tìm kiếm theo cả hai hướng (tiền và lùi). Điều này là do BFS đảm bảo các cây tìm kiếm phát triển đồng đều, tăng khả năng gặp nhau ở giữa không gian trạng thái, và tránh được nguy cơ không tìm ra lời giải nếu sử dụng tìm kiếm theo chiều sâu (DFS).

2.2.2 Tại sao không nên sử dụng DFS trong Bidirectional

2.2.2.1 Nguy cơ không tìm ra lời giải (Không đầy đủ)

- **Nguyên tắc gặp nhau:** Tim kiếm hai chiều hoạt động dựa trên nguyên tắc hai cây tìm kiếm (một từ nút xuất phát, một từ nút đích) phát triển đồng thời và gặp nhau tại một nút chung.
- **Hạn chế của DFS:** Tim kiếm theo chiều sâu (DFS) có nguyên tắc là lựa chọn nút sâu nhất (xa nút gốc nhất) để mở rộng trước. Thuật toán này sẽ di chuyển theo một nhánh cho tới nút sâu nhất (nút không có nút con), trước khi chuyển sang nhánh tiếp theo.
- **Rủi ro bế tắc:** Nếu một trong hai quá trình tìm kiếm (tiền hoặc lùi) đi vào một nhánh vô hạn hoặc một nhánh không chứa lời giải mà không gặp nhánh tìm kiếm của phía bên kia, thuật toán có thể không cho phép tìm ra lời giải.
- **Kết luận:** DFS có thể không đầy đủ (không đảm bảo tìm ra lời giải nếu có) nếu không gian trạng thái là vô hạn, vì nó có thể di chuyển theo một đường đi không chứa nút đích và có độ sâu vô hạn mà không chuyển sang nhánh khác được. Nguy cơ này càng lớn khi áp dụng cho cả hai chiều tìm kiếm, vì mỗi nhánh độc lập có thể "lạc lối" mà không gặp nhau.

2.2.2.2 Vấn đề vòng lặp

- **Vòng lặp trong DFS:** Nếu không gian trạng thái chứa vòng lặp, tìm kiếm theo chiều sâu sẽ lặp vô hạn mà không tìm được lời giải.
- **BFS khắc phục Vòng lặp:** Ngược lại, BFS, ngay cả khi có vòng lặp, vẫn tìm ra lời giải, mặc dù phải tính toán lâu hơn và xem xét nhiều trạng thái hơn.
- **Yêu cầu kỹ thuật:** Để tránh vòng lặp khi dùng DFS, cần phải duy trì danh sách nút đóng, điều này đòi hỏi nhiều bộ nhớ.

Tóm lại, việc sử dụng BFS trong Tim kiếm hai chiều là bắt buộc vì BFS đảm bảo rằng cả hai cây tìm kiếm phát triển theo từng lớp độ sâu, tăng khả năng gặp nhau ở giữa không gian trạng thái, nhờ đó đảm bảo tính đầy đủ và tối ưu (về độ sâu lời giải nông nhất).

2.2.3 Một số biến thể của Bidirectional

Bảng 2. 1: So sánh các biến thể của Tìm kiếm Hai chiều

Biến Thể	Đồ Thị	Hàm Heuristic	Ưu Điểm	Nhược Điểm & Thách thức
Bidirectional BFS	Không trọng số	Không	Đơn giản, hiệu quả cho đồ thị không trọng số	Không xử lý được chi phí, khó áp dụng cho đồ thị có hướng
Bidirectional UCS	Có trọng số	Không	Tìm được đường đi ngắn nhất (chi phí thấp nhất)	Chiến lược dừng phức tạp hơn
Bidirectional A*	Có trọng số	Có	Nhanh hơn nhờ định hướng của heuristic	Rất phức tạp, đòi hỏi heuristic chất lượng cao và chiến lược kết hợp tối ưu

2. 1 là các biến thể của thuật toán Tìm Kiếm 2 Chiều cũng như so sánh ưu và nhược điểm của chúng.

2.3 Phân tích độ phức tạp của thuật toán, Ưu và Nhược điểm khi so sánh với các thuật toán khác

Các biến thể của Bidirectional Search mở rộng đáng kể phạm vi ứng dụng của nó. Từ những đồ thị đơn giản không trọng số, nó có thể được điều chỉnh để giải quyết các bài toán tối ưu hóa phức tạp thậm chí. Để phân tích độ phức tạp, ta sử dụng các tham số tiêu chuẩn:

- b : hệ số nhánh trung bình (số nút con trung bình của một nút).
- d : độ sâu của lời giải (từ start đến goal).
- m : độ sâu tối đa của không gian tìm kiếm.

2.3.1 Phân tích độ phức tạp thuật toán

2.3.1.1 Độ Phức Tạp Thời Gian

Đối với Bidirectional BFS (sử dụng cho đồ thị không trọng số), mỗi phía (thuận và nghịch) chỉ cần đi sâu đến $d/2$. Do đó, số nút cần duyệt ở mỗi phía là $O(b^{d/2})$.

Tổng độ phức tạp thời gian là:

$$O(b^{d/2} + b^{d/2}) = O(b^{d/2})$$

So sánh với BFS một chiều có độ phức tạp $O(b^d)$, Bidirectional Search giảm độ phức tạp thời gian theo hàm mũ.

2.3.1.2 Độ Phức Tạp Không Gian

Bidirectional BFS cũng có độ phức tạp không gian là $O(b^{d/2})$, do mỗi phía cần lưu trữ một hàng đợi và một tập đã duyệt có kích thước cỡ $b^{d/2}$.

So sánh với BFS một chiều có độ phức tạp không gian $O(b^d)$, Bidirectional Search cũng giảm độ phức tạp không gian theo hàm mũ.

2.3.2 So sánh với các Thuật toán Tìm kiếm Khác

Dưới đây là bảng so sánh Bidirectional Search với các thuật toán tìm kiếm phổ biến.

Bảng 2. 2: So sánh các thuật toán tìm kiếm

Thuật Toán	Độ Phức Tạp Thời Gian	Độ Phức Tạp Không Gian	Tính Tối Ưu
Depth-First Search (DFS)	$O(b^m)$	$O(m)$	Không
Breadth-First Search (BFS)	$O(b^d)$	$O(b^d)$	Có
Uniform-Cost Search (UCS)	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	Có
A*	$O(b^d)$	$O(b^d)$	Có (với heuristic phù hợp)
Bidirectional BFS	$O(b^{d/2})$	$O(b^{d/2})$	Có

2.3.3 Ưu điểm và Nhược điểm

- **Ưu điểm chính của Bidirectional Search:** Hiệu suất vượt trội cả về thời gian và bộ nhớ so với BFS và A* trong các bài toán phù hợp.
- **Nhược điểm:** Khó áp dụng nếu không xác định được chiều tìm kiếm ngược (ví dụ: trong một số đồ thị có hướng).

2.3.4 Kết luận

Bidirectional Search là một cải tiến mạnh mẽ so với các thuật toán tìm kiếm kinh điển. Nó giảm cả độ phức tạp thời gian và không gian theo cấp số nhân, từ $O(b^d)$ xuống còn $O(b^{d/2})$. Tuy nhiên, việc áp dụng thuật toán này cần cân nhắc đến đặc điểm của bài toán, đặc biệt là khả năng tìm kiếm ngược từ trạng thái đích.

Việc lựa chọn thuật toán tìm kiếm phụ thuộc vào từng bài toán cụ thể:

- Nếu không gian tìm kiếm lớn và biết rõ điểm bắt đầu/kết thúc: Ưu tiên Bidirectional Search.
- Nếu có hàm heuristic tốt: Cân nhắc sử dụng A*.
- Nếu ưu tiên tiết kiệm bộ nhớ: DFS có thể là lựa chọn phù hợp.

2.4 Nhược điểm và thách thức kỹ thuật

- **Xác định đích:** Cần biết trạng thái đích rõ ràng (không phù hợp với bài toán "tối ưu hóa").
- **Quản lý bộ nhớ:** Phải lưu trữ đồng thời hai Frontier và tập visited.
- **Điểm gặp nhau:** Cần cơ chế so khớp hiệu quả (hashing, bảng băm).
- **Khó áp dụng:** Với đồ thị có cạnh trọng số khác nhau $\rightarrow$ cần biến thể Dijkstra hoặc A* hai phía.

2.5 Ứng dụng và hạn chế trong thực tế

Sau khi đã nắm vững nguyên lý và hiệu suất vượt trội trên lý thuyết, một câu hỏi đặt ra là: Bidirectional Search được áp dụng như thế nào trong thực tế và liệu nó có phải là "chìa khóa vạn năng"? Phần tiếp theo sẽ lần lượt khám phá những ứng dụng thiết thực của thuật toán trong các lĩnh vực then chốt, đồng thời cũng phân tích những hạn chế cố hữu – những bài toán mà nó tỏ ra kém hiệu quả hoặc không thể áp dụng được. Sự hiểu biết toàn diện này là chìa khóa để các kỹ sư và nhà khoa học dữ liệu đưa ra quyết định sáng suốt khi lựa chọn giải thuật phù hợp nhất cho từng tình huống cụ thể.

2.5.1 Ứng dụng trong thực tế

- Tìm đường trong bản đồ số: Google Maps, GPS Navigation.
- Trò chơi & giải đố: Rubik's Cube, 8-Puzzle.
- Mạng máy tính: tìm đường đi ngắn nhất giữa hai node trong routing.
- Robotics: dẫn đường cho robot từ vị trí hiện tại đến mục tiêu đã biết.
- Công cụ tìm kiếm: tối ưu quá trình so khớp trong database.

2.5.2 Hạn chế trong thực tế

- **Trạng thái đích không rõ ràng:** ví dụ "tìm trạng thái tối ưu" thay vì một đích duy nhất.
- **Đồ thị trọng số phức tạp:** khó áp dụng trực tiếp, cần biến thể A* hai phía.
- **Khó khăn trong không gian trạng thái lớn:** chi phí bộ nhớ vẫn tăng nhanh, dù ít hơn BFS.
- **Tổ chức dữ liệu phức tạp:** để kiểm tra giao nhau nhanh cần kỹ thuật hashing tốt.

2.6 Kết luận

Như vậy, Chương 2 đã trình bày một cách có hệ thống cơ sở lý thuyết của thuật toán Tìm kiếm Hai chiều (Bidirectional Search). Có thể khẳng định đây là một trong những chiến lược tìm kiếm mà hiệu quả nhất khi bài toán xác định rõ điểm xuất phát và điểm đích. Sức mạnh cốt lõi của phương pháp này nằm ở việc giảm độ phức tạp tính toán cả về thời gian và không gian từ $O(b^d)$ xuống còn $O(b^{d/2})$, một cải tiến theo hàm mũ so với các thuật toán tìm kiếm một chiều kinh điển như BFS hay DFS.

Ưu điểm này đạt được nhờ cơ chế tìm kiếm đồng thời từ cả hai phía và gặp nhau tại điểm trung gian, với điều kiện tiên quyết là phải sử dụng BFS cho cả hai hướng để đảm bảo tính đầy đủ và tối ưu. Tuy nhiên, thuật toán cũng bộc lộ những hạn chế không thể bỏ qua, đặc biệt là yêu cầu về một trạng thái đích rõ ràng và những thách thức trong việc áp dụng cho đồ thị có hướng hoặc có trọng số phức tạp - nơi đòi hỏi các biến thể như Bidirectional UCS hay Bidirectional A*.

Những kiến thức nền tảng này chính là tiền đề quan trọng để tiếp tục khám phá các ứng dụng thực tế cũng như đánh giá hiệu quả của thuật toán trong các tình huống thực tế, sẽ được trình bày chi tiết trong các chương tiếp theo.

CHƯƠNG 3. LẬP TRÌNH VÀ THỰC NGHIỆM THUẬT TOÁN BIDIRECTIONAL SEARCH